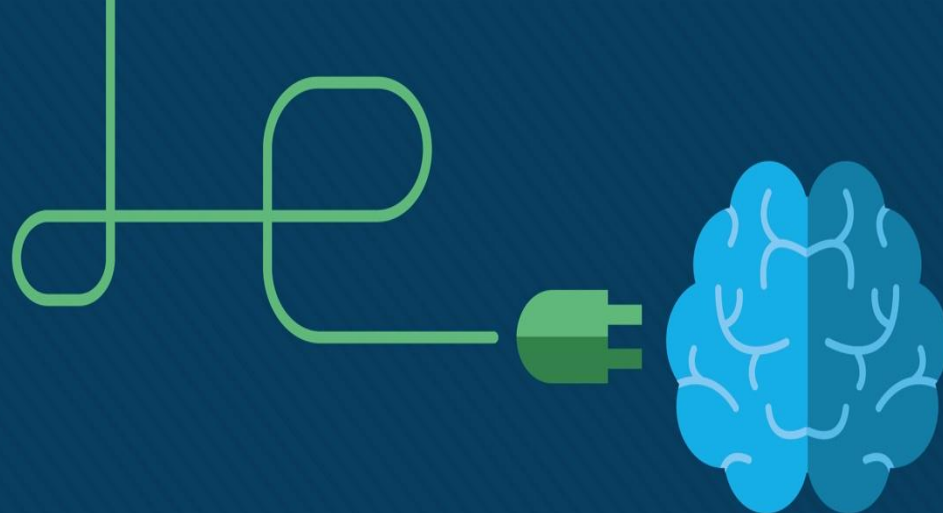


Opsamling

- Er der opstået spørgsmål til OSPF?



Module 4: ACL Concepts

Module 5: ACLs for IPv4 Configuration

Module 6: NAT for IPv4

Enterprise Networking, Security, and Automationv7.0
(ENSA)



Indhold

- Hvad er en ACL?
- Hvad bruger vi ACL til?
- Hvor placerer vi ACLs?
- Konfiguration af ACLs
- NAT
- Konfiguration af NAT

Ca. 37 dias

Hvad er en Access Control List (ACL)?

En ACL er en liste af parametre der, hvis en pakke matcher de parametre, skal gøre et eller andet med pakken, f.eks. give adgang eller afvise pakken der er på vej ind eller ud af et router interface..

ACL skal konfigureres og aktiveres før den træder i kraft. Ellers vil alle pakker accepteres, så længe der er forbindelse (*indtil nu har I bare slet ikke opdaget at interfacene undersøger om der skulle være en ACL, som pakkerne skal checkes op imod 😊*)

Der kan matches parametre baseret på:

- Afsender adresse (source)
- Modtager adresse (destination)
- TCP & UDP port nummer

Der kan skrives ACLs til alle L3 protokoller.

Hvad skal vi med Access Control Lister?

- Man bruger altså (bland andet) ACLs til at filtrere datapakker, ud fra nogle parametre
- Eksempelvis bruger man ACLs til at indkredse et adresseområde, som skal have adgang til at blive oversat til andre adresser, når de passerer en router.
- Det er det vi normalt kalder NAT (Network Address Translation).
- De adresser der IKKE matcher ACL'en, bliver ikke oversat!
- Man bruger også ACLs til at stoppe trafik man ikke vil have ind eller ud gennem et interface på routeren. Måske vil man KUN acceptere #80 trafik at komme ind til webserveren fra Internettet?

Match eller ej?

For at afgøre om en pakke matcher en ACL, bliver pakkerne testet gennem ACLs betingelser i sekventiel orden.

Når en betingelse “matcher,” er der ikke flere betingelser der bliver analyseret.

Inden vi konfigurerer – vigtigt at vide:

Der er en underforstået/usynlig “deny any” (afvis alle) betingelse i enden af ACL.

=> Hvis en pakke ikke matcher nogle af betingelserne i ACL, bliver den droppet

Ikke-videnskabeligt eksempel ☺ :

Access Control Liste 112:

- Vi tillader røde biler med 4 hjulstræk at komme gennem gaden
- Vi nægter blå biler adgang
- Vi tillader biler med blå blink, uanset bilens farve, adgang
- *Vi nægter alt der ikke specifikt er angivet herover*

Hjælp mig.... hvem får lov at komme gennem gaden?

- En grøn bil med blå blink?

Ja – rammes af linje 3, ikke nogen tidligere.

- En gul bil med 4 hjulstræk uden blå blink?

Nej – rammes af implicit "deny any"

- En blå bil med blå blink?

Nej – rammes først af linje 2, så der læses ikke videre.

I virkeligheden kan en ACL f.eks. se sådan her ud:

- Skriv ACL betingelserne sekventiel i global konfigurations mode.

```
Router(config) #access-list access-list-number  
{permit/deny} {(source address)}
```

```
Router(config) #access-list 1 deny 192.5.5.10 0.0.0.0
```

```
Router(config) #access-list 1 permit any
```

- Aktivér ACL på et eller flere interfaces:

```
Router(config-if) #{protocol} access-group  
access-list-number {in/out}
```

```
Router(config-if) #ip access-group 1 out
```

Nummerering af ACLs

Der er flere typer ACL.

access-list-number specificer typen.

ACL Type	ACL Nummer
IP Standard	1 til 99 og 1300 til 1999
IP Extended	100 til 199 og 2000 til 2699
AppleTalk	600 til 699
IPX Standard	800 til 899
IPX Extended	900 til 999
IPX SAP	1000 til 1099

```
Router(config)#access-list [access-list-number]  
{permit/deny} {test-conditions}
```

{test conditions} = er det der skal matche for at pakken
"permit'es" eller "deny'es"

Det kan f.eks. være en bestemt afsender/source.

En source adresse kan være et subnet, et område af adresser,
eller en host.

"Wildcard masken" fortæller routeren hvilke bits af IP masken
der skal tjekkes. Se de følgende slides.

IP mask Wildcard mask


```
Lab-A(config)#access-list 1 deny 192.5.5.10 0.0.0.0
```

Grundlæggende (Standard ACL)

- Skriv ACL betingelserne sekventiel i global konfigurations mode.

```
Router(config) #access-list access-list-number  
{permit/deny} {(source address)}
```

```
Router(config) #access-list 1 deny 192.5.5.10 0.0.0.0
```

```
Router(config) #access-list 1 permit any
```

- Aktivér ACL på et eller flere interfaces:

```
Router(config-if) #{protocol} access-group  
access-list-number {in/out}
```

```
Router(config-if) #ip access-group 1 out
```

Eksempler:

På de næste slides, vil vi lave nogle eksempler på wildcard masking. **Bliv ikke nervøs hvis du ikke forstår det med det samme.** Som med subnetting, **øvelse gør mester.**

Skriv en ip adr. og en wildcard mask til check af trafik fra alle hoster i netværket: 192.5.5.0 255.255.255.0

Svar: 192.5.5.0 0.0.0.255

Eksempler:

Skriv ip adresse og wildcard maske for check af alle hoster i subnettet: 192.5.5.32 255.255.255.224

Hvis du svarede 192.5.5.32 0.0.0.31 HAR DU RET!!

Skriv wildcard mask for subnettet 172.16.128.0 med en subnet mask på 255.255.128.0?

Svar: 172.16.128.0 0.0.127.255

Skriv wildcard mask for subnettet 172.16.16.0 med en subnet mask på 255.255.252.0?

Svar: 172.16.16.0 0.0.3.255

Nu skulle du have forstået hvordan det der med wildcard mask hænger sammen med subnet mask. Hvis ikke prøv igen!!! 😊

Eksempler:

Det kan blive nødvendigt at udelukke en del af et (sub)netværk, samtidig med at man tillader en anden del.

For at "indkredse" et område i et subnet, er det ofte nødvendigt at arbejde på et binært niveau.

For eksempel:

Vi har et netværk: 192.5.5.0 255.255.255.0

Kursisterne benytter adresse-området 192.5.5.0 til 192.5.5.127

Lærerne benytter området 192.5.5.128 til 192.5.5.255.

Hvordan vil du skrive netværks-id og wildcard mask, der deler netværkets adresser op i et område for kursisterne og et område for lærerne?

Hvordan skriver vi selve ACL'en, der giver kursisterne adgang til en server og spærrer for lærerne?

Spar tid: host kommandoen

Mange netværks administrator ønsker at skrive en ACL der giver permit (eller deny) fra en bestemt host. Betingelserne kan skrives på to måder. Enten...

```
Lab-A(config)#access-list 1 permit 192.5.5.10 0.0.0.0
```

Eller....

```
Lab-A(config)#access-list 1 permit host 192.5.5.10
```

Spar tid: any kommandoen

Den samme "genvej" kan man bruge her:

```
Lab-A(config)#access-list 1 permit 0.0.0.0 255.255.255.255
```

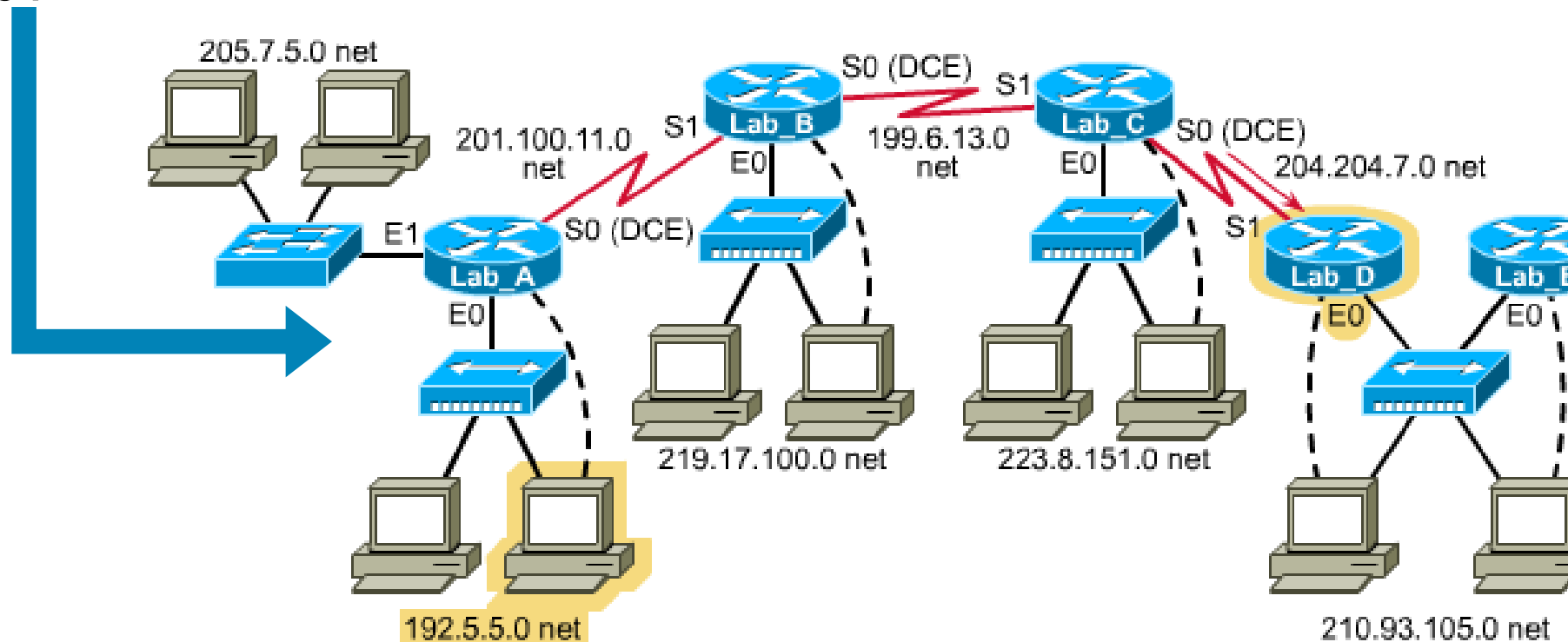
Eller.....

```
Lab-A(config)#access-list 1 permit any
```

Korrekt placering af Standard ACL

Standard ACL har ikke en destination parameter. Du bliver derfor nødt til at placere standard ACL så tæt på modtageren som muligt (i dette tilfælde f.eks. Lab_D's LAN).

For at se hvorfor, så hvad vil ske med alt ip trafik hvis du placerede en "deny 192.5.5.0 0.0.0.255" betingelse på Lab-A's E0?



Lille øvelse (30 minutter)

ACL med instruktion – 4.1.4 ACL demo i Learn

- *Access control list (ACL) bruges i denne øvelse til at forhindre en ping i at nå frem til enheder på remote netværk.*
- *Efter fjernelse af ACL i konfigurationen, kommer ping pakker fint igennem.*

Extended ACL Oversigt

Extended ACL er numrene fra 100 - 199 og “udvider” funktionaliteten i forhold til Standard ACL (= *kun source adr.*).

Udvidelsen inkluderer egenskaben at filtrere trafik baseret på...

- Destination adresse
- Protokol

Du kan skrive betingelser til kun at udelukke (deny) protokoller som “tftp” eller “http”

Du kan benytte operatorer som eq, gt, lt, og neq (lig med, større end, mindre end, og ikke lig med) til at specificere hvordan en bestemt protokol skal benyttes.

For eksempel, hvis du ønsker en access liste der tillader al trafik på nær http pakker, kan du skrive

```
permit ip any any neq 80
```

Grundlæggende (Extended ACL)

Skriv ACL betingelserne sekventiel i global configurations mode.

```
Router(config)# access-list access-list-number  
{permit|deny} {protocol|protocol-keyword} {source  
source-wildcard} {destination destination-wildcard}  
[protocol-specific options] [log]
```

```
Router(config)#access-list 101 deny tcp 192.5.5.0  
0.0.0.255 210.93.105.0 0.0.0.255 eq telnet log
```

Aktivér ACL på et interface i rigtig retning:

```
Router(config-if)#{protocol} access-group access-  
list-number {in/out}
```

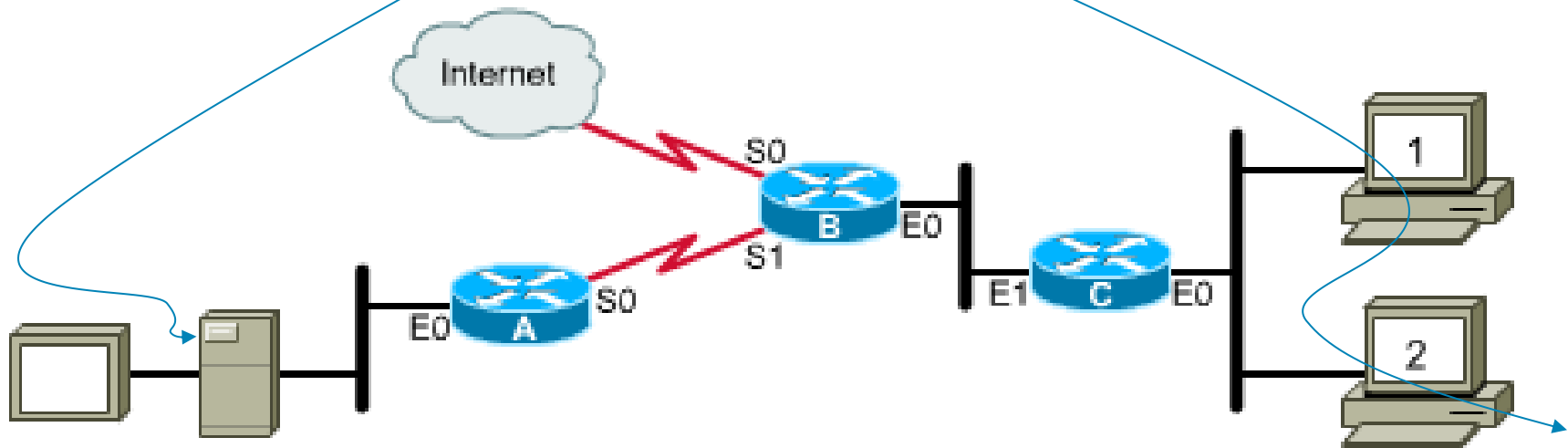
```
Router(config-if)#ip access-group 101 out
```

Korrekt placering af Extended ACL

I grafikken herunder, ønsker vi at hindre enheder på netværk 221.23.123.0 i at tilgå server 198.150.13.34.

Hvilken router og interface skal access listen aktiveres på på?
Og skal det være ind- eller udgående retning?

Router C, på E0 - indgående (tættest på afsender!)



Server
IP: 198.150.13.34

Router A
E0: 198.150.13.56
S0: 223.11.34.123

Router B
E0: 222.21.38.100
S0: 220.100.5.122
S1: 223.11.34.124

Router C
E0: 221.23.123.96
E1: 222.21.38.123

Workstation 1
IP: 221.23.123.45

Workstation 2
IP: 221.23.123.40

ACLs på vty linjer

The virtual terminal or “VTY” lines are virtual lines that allow connecting to the device using telnet or Secure Shell (SSH).

Cisco - Hyperterminal

Creating the standard list:

```
Rt1(config)#access-list 2 permit 172.16.1.0 0.0.0.255  
R11(config)#access-list 2 permit 172.16.2.0 0.0.0.255  
R11(config)#access-list 2 deny any
```

Applying the access list:

```
Rt1(config)#line vty 0 4  
Rt1(config)#login  
Rt1(config)#password secret  
Rt1(config)#access-class 2 in
```

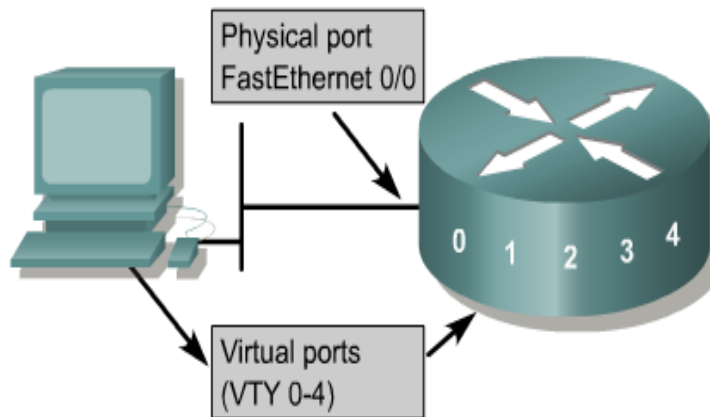


Hvor træls er det lige???

Access-list?

Access-class?

Ikke at forglemme Access-group ☺



Navngive ACL

Der er muligheden for at navngive ACL.

Når du har navngivet en ACL, ændres prompten og du behøver ikke længer at skrive **access-list** og **access-list-number** parameter.

Eksempel :

```
R1 (config) #ip access-list standard over_and
R1 (config-std-nacl) #deny host 192.5.5.10
R1 (config-std-nacl) #exit
R1 (config) #interface g0/0
R1 (config-if) #ip access-group over_and out
```

```
(config-std-nacl)
```

```
      ↑      ↑
standard  named ACL
```



Editing ACLs

```
R1(config)#access-list 1 deny host 192.168.10.99
R1(config)#access-list 1 permit 192.168.0.0 0.0.255.255
```

```
R1#show access-lists 1
Standard IP access list 1
 10 deny 192.168.10.99
 20 permit 192.168.0.0, wildcard bits 0.0.255.255
R1#
```

```
R1#conf t
R1(config)#ip access-list standard 1
R1(config-std-nacl)#no 10
R1(config-std-nacl)#10 deny host 192.168.10.10
R1(config-std-nacl)#end
R1#
```

```
R1#show access-lists
Standard IP access list 1
 10 deny 192.168.10.10
 20 permit 192.168.0.0, wildcard bits 0.0.255.255
...
```

Ingen sekvensnumre hvis
man bruger kommandoen
#show access-list 1

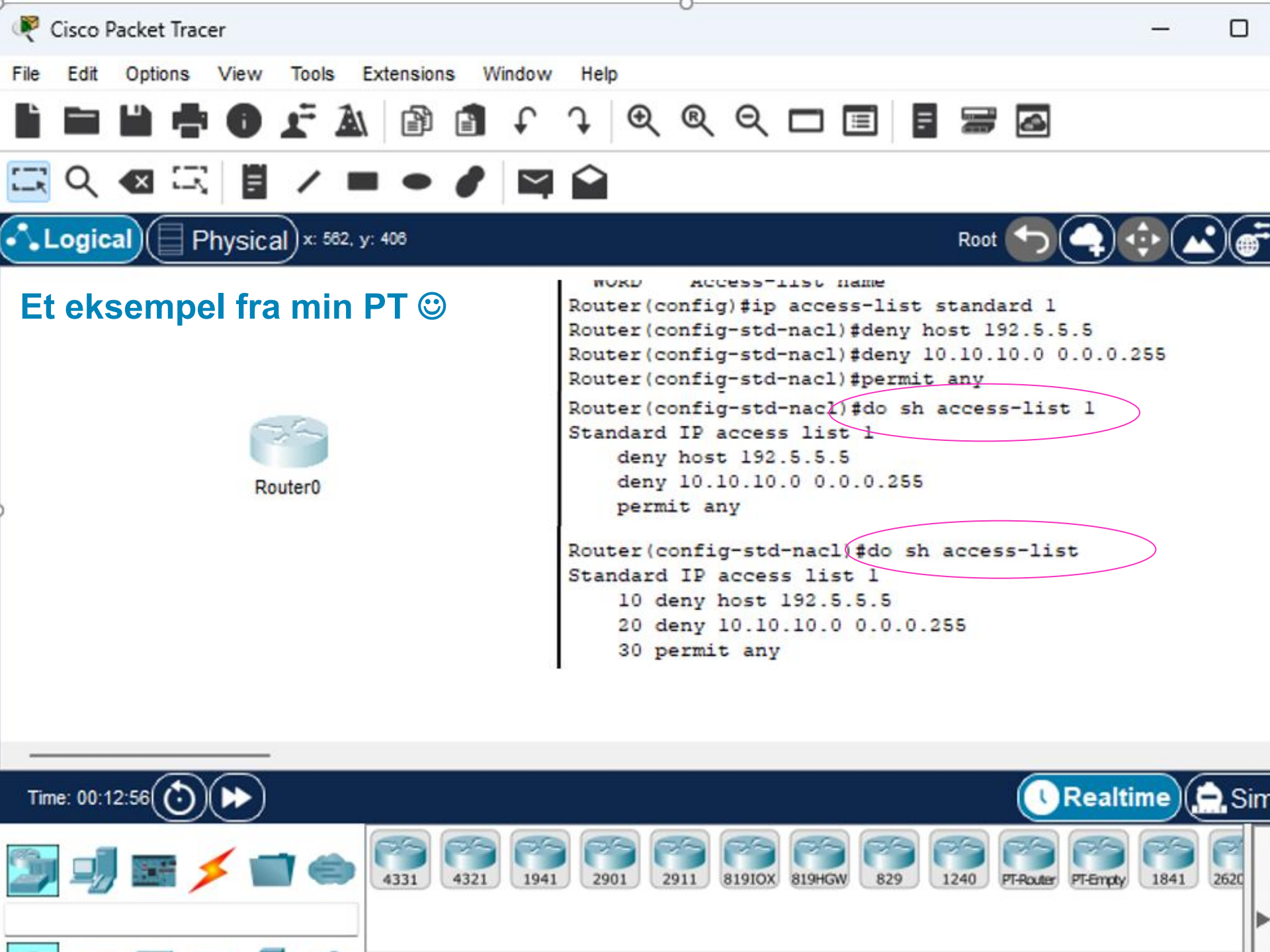


```
Router#sh access-list 1
Standard IP access list 1
  deny host 192.168.10.99
  permit 192.168.0.0 0.0.255.255
```

```
Router#sh access-list
Standard IP access list 1
 10 deny host 192.168.10.99
 20 permit 192.168.0.0 0.0.255.255
```

Om det kun gælder for PT
eller også på det fysiske
udstyr, er jeg i tvivl om ☺

Men med kommandoen
#show access-lists .



Et eksempel fra min PT ☺



```
WORD ACCESS-LIST NAME
Router(config)#ip access-list standard 1
Router(config-std-nacl)#deny host 192.5.5.5
Router(config-std-nacl)#deny 10.10.10.0 0.0.0.255
Router(config-std-nacl)#permit any
Router(config-std-nacl)#do sh access-list 1
Standard IP access list 1
deny host 192.5.5.5
deny 10.10.10.0 0.0.0.255
permit any

Router(config-std-nacl)#do sh access-list
Standard IP access list 1
10 deny host 192.5.5.5
20 deny 10.10.10.0 0.0.0.255
30 permit any
```

Time: 00:12:56

Realtime

Sim



Kontrollere ACL

- Show commandoer:

`show access-lists`

viser alle access-lists konfigureret på routeren

`show access-lists {name | number}`

viser en bestemt access liste

`show ip interface`

viser hvilken ACL der er aktiv ind- og udgående

`show running-config`

Hvis vi ikke lige har haft pause 😊
Inden vi går videre med NAT



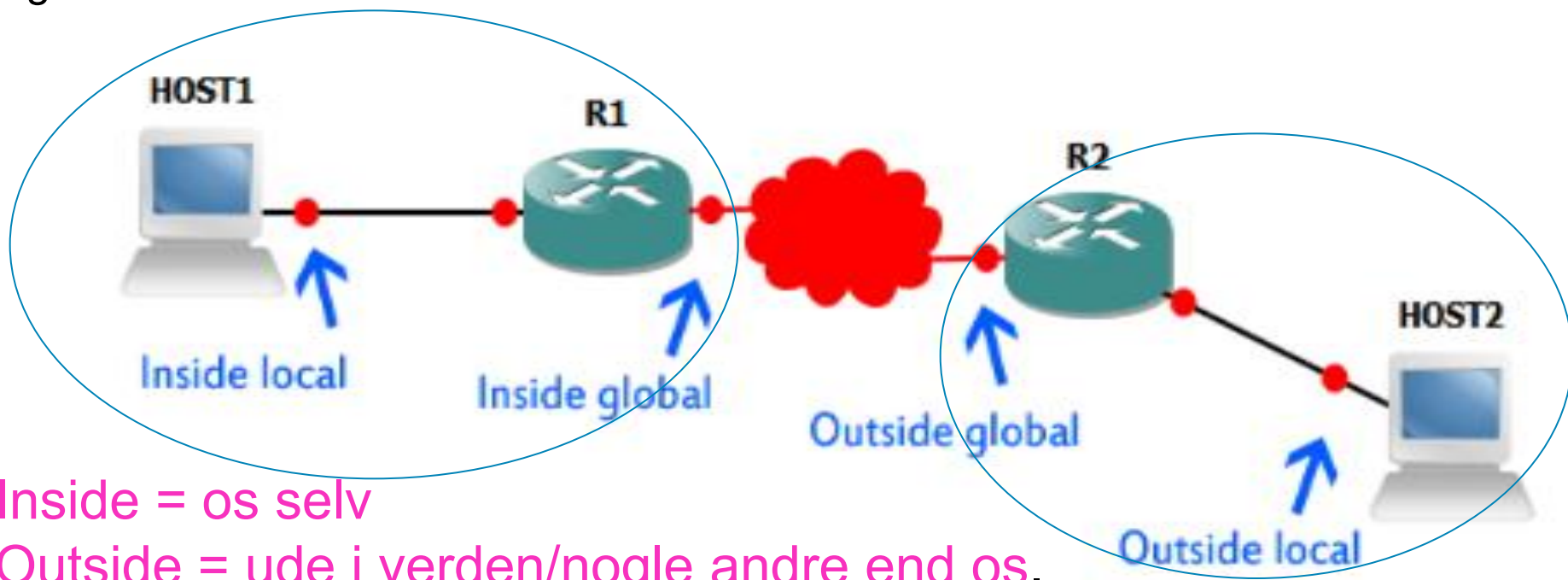
With NAT, Cisco defines 4 different types of addresses as follows:

Inside local address – Den (private) IP adresse en host har på vores eget LAN. Kan ikke ses af “fremmede” (fra andre netværk).

Inside global address – En offentlig IP adresse som repræsenterer indersiden af vores LAN. Den adresse som “fremmede” kan se.

Outside global address – Den offentlige IP adresse som repræsenterer et andet netværk. Den adresse vi kan se.

Outside local address – Den (private) IP adresse en fremmed host har på dens eget netværk. Vi kan ikke se den!



Inside = os selv

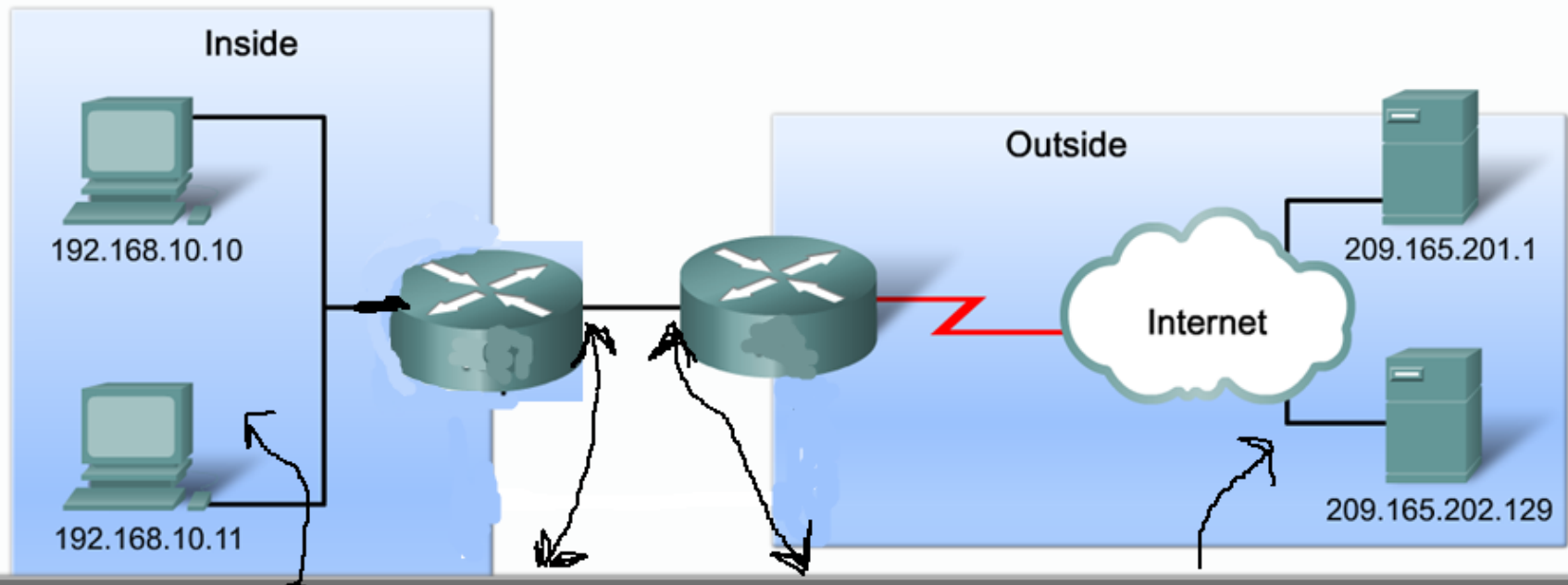
Outside = ude i verden/nogle andre end os.

■ NAT and NAT overload

NAT = én-til-én oversættelse

PAT (Port Address Translation = NAT overload) = mange-til-én (eller nogle få) oversættelse.

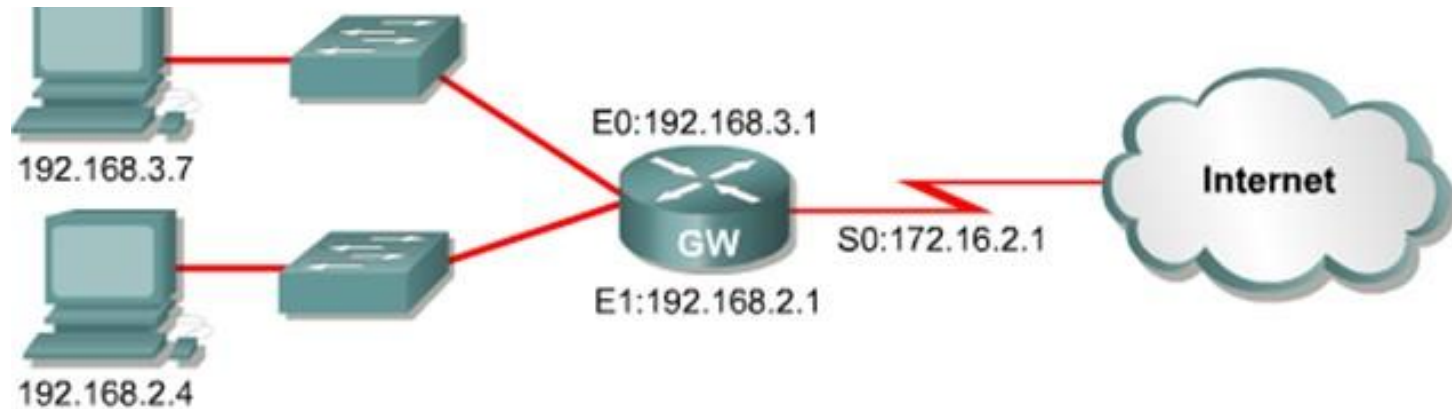
Det kigger vi på i de næste dias.



NAT Table with Overload

Inside Local IP Address	Inside Global IP Address	Outside Global IP Address	Outside Local IP Address
192.168.10.10:1555	209.165.200.226:1555	209.165.201.1:80	209.165.201.1:80
192.168.10.11:1331	209.165.200.226:1331	209.165.202.129:80	209.165.202.129:80

Port Address Translation = den vi “altid” bruger ☺



```
interface ethernet 0
  ip address 192.168.3.1 255.255.255.0
  ip nat inside
!
interface ethernet 1
  ip address 192.168.2.1 255.255.255.0
  ip nat inside
!
interface serial 0
  ip address 172.16.2.1 255.255.255.0
  ip nat outside
!
ip nat inside source list 1 interface serial 0 overload
!
access-list 1 permit 192.168.2.0 0.0.0.255
access-list 1 permit 192.168.3.0 0.0.0.255
```

HUSK “overload”
nøgleordet!!!



Denne konf. er med én global adresse.

Man kan også oprette en pulje af adresser (find selv eksempel om nødvendigt)

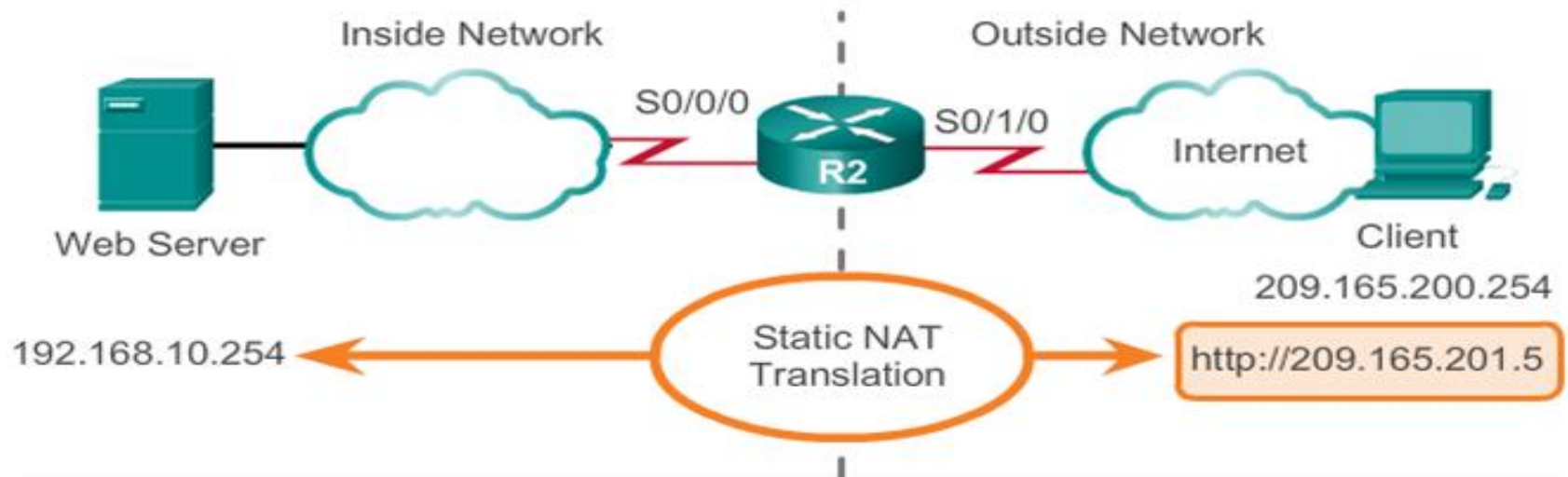
3 enkle trin for at konfigurere NAT 😊

- 1) Definer adresser eller adresse-områder, der skal "oversættes" (udskiftes/NAT'es), vha ACL.
- 2) Aktivér NAT (#ip nat inside osv)
- 3) Definér hvilke interfaces der er hhv. inderside og yderside i forhold til NAT

Statisk NAT

En intern adresse vil altid blive oversat til den samme offentlige adresse!

I hvilke tilfælde kunne det være smart???



Establishes static translation between an inside local address and an inside global address.

```
R2(config)# ip nat inside source static 192.168.10.254 209.165.201.5
```

```
R2(config)# interface Serial0/0/0
```

```
R2(config-if)# ip address 10.1.1.2 255.255.255.252
```

Identifies interface serial 0/0/0 as an inside NAT interface.

```
R2(config-if)# ip nat inside
```

```
R2(config-if)# exit
```

```
R2(config)# interface Serial0/1/0
```

```
R2(config-if)# ip address 209.165.200.225 255.255.255.224
```

Identifies interface serial 0/1/0 as the outside NAT interface.

```
R2(config-if)# ip nat outside
```

Verifying Translations

```
R2# show ip nat translations
```

Pro	Inside global	Inside local	Outside local	Outside global
tcp	209.165.200.226:51839	192.168.10.10:51839	209.165.201.1:80	209.165.201.1:80
tcp	209.165.200.226:42558	192.168.11.10:42558	209.165.202.129:80	209.165.202.129:80

```
R2#
```

```
R2# debug ip nat
```

```
IP NAT debugging is on
```

```
R2#
```

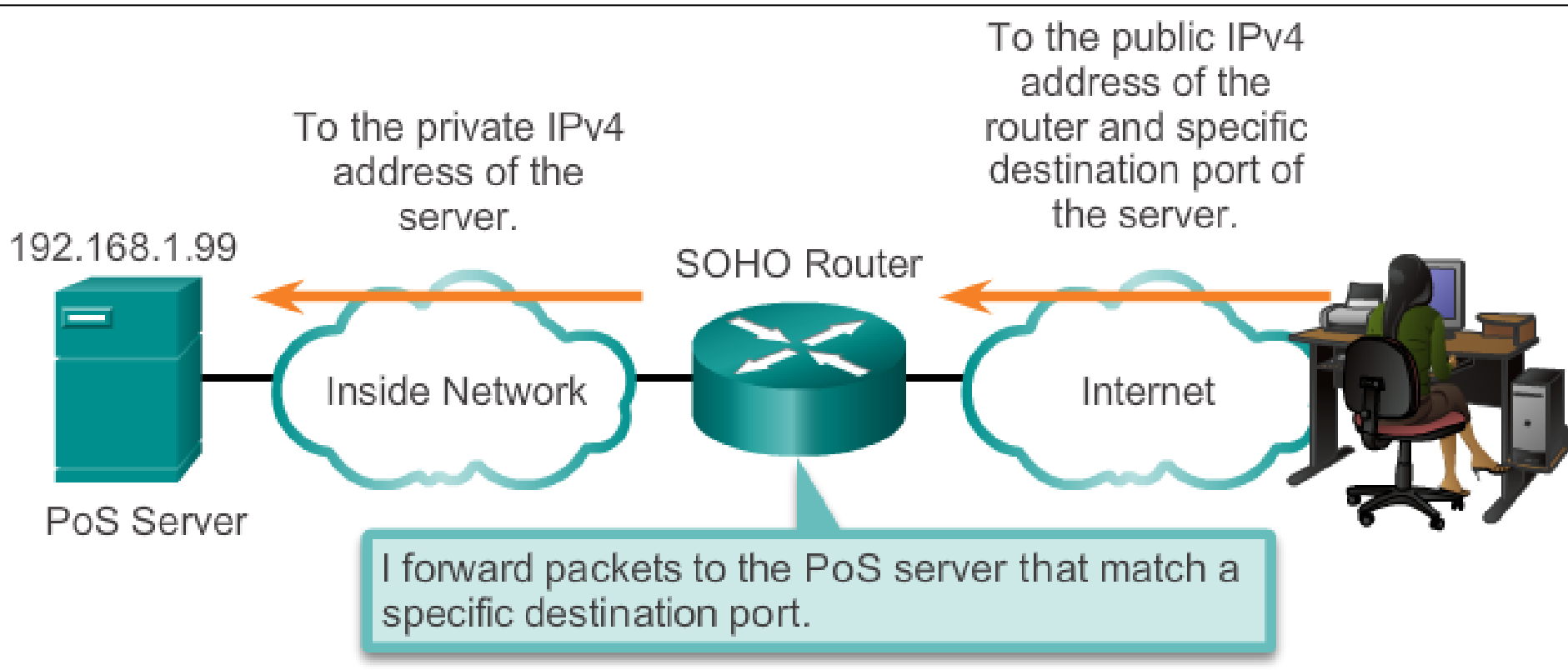
```
*Oct  6 19:55:31.579: NAT*: s=192.168.10.10->209.165.200.225, d=209.165.200.254 [14434]
*Oct  6 19:55:31.595: NAT*: s=209.165.200.254, d=209.165.200.225->192.168.10.10 [6334]
*Oct  6 19:55:31.611: NAT*: s=192.168.10.10->209.165.200.225, d=209.165.200.254 [14435]
*Oct  6 19:55:31.619: NAT*: s=192.168.10.10->209.165.200.225, d=209.165.200.254 [14436]
*Oct  6 19:55:31.627: NAT*: s=192.168.10.10->209.165.200.225, d=209.165.200.254 [14437]
*Oct  6 19:55:31.631: NAT*: s=209.165.200.254, d=209.165.200.225->192.168.10.10 [6335]
*Oct  6 19:55:31.643: NAT*: s=209.165.200.254, d=209.165.200.225->192.168.10.10 [6336]
*Oct  6 19:55:31.647: NAT*: s=192.168.10.10->209.165.200.225, d=209.165.200.254 [14438]
*Oct  6 19:55:31.651: NAT*: s=209.165.200.254, d=209.165.200.225->192.168.10.10 [6337]
*Oct  6 19:55:31.655: NAT*: s=192.168.10.10->209.165.200.225, d=209.165.200.254 [14439]
*Oct  6 19:55:31.659: NAT*: s=209.165.200.254, d=209.165.200.225->192.168.10.10 [6338]
```

```
<Output omitted>
```

- Portforwarding er "ekstra", så **spring til dias 39 og opgaven.**
- Hvis nogen er nysgerrige på portforwarding, kan de selv kigge på de næste par dias.

Port Forwarding

- Hvad er port forwarding?????



Port Forwarding

Når trafik starter udefra (Internettet/et andet netværk) og vil ind på vores netværk, ved routeren ikke umiddelbart hvor den skal sende pakkerne hen. **Pakkerne sendes jo til vores globale/offentlige IP adresse.**

Port forwarding fortæller routeren, hvor den skal sende pakkerne hen. Altså kobler bestemt type af trafik med bestemt IP adresse på LAN (eller DMZ)

LINKSYS®

Filters **Forwarding** Dynamic Routing Static Routing DMZ Host MAC Addr. Clone Setup

PORT RANGE FORWARDING

Port forwarding can be used to set up public services on your network. When users from the Internet make certain requests on your router, they will be redirected to the specified IP.

Customized Applications		Ext.Port	Protocol	Protocol	IP Address	Enable
			TCP	UDP		
		80 To 80	☒	☐	192.168.1.100	☒
		0 To 0	☐	☐	192.168.1.0	☐
		0 To 0	☐	☐	192.168.1.0	☐
		0 To 0	☐	☐	192.168.1.0	☐
		0 To 0	☐	☐	192.168.1.0	☐
		0 To 0	☐	☐	192.168.1.0	☐
		0 To 0	☐	☐	192.168.1.0	☐
		0 To 0	☐	☐	192.168.1.0	☐
		0 To 0	☐	☐	192.168.1.0	☐
		0 To 0	☐	☐	192.168.1.0	☐
		0 To 0	☐	☐	192.168.1.0	☐

UPnP Forwarding Port Triggering

Apply Cancel

Et andet eksempel (jeg ved det er gammelt!) Port Forwarding

Forwards

Application	Port from	Protocol	IP Address	Port to	Enable
emule	48913	TCP ▼	192.168.1.201	48913	☐
emule2	6047	UDP ▼	192.168.1.201	6047	☐
wol	9	UDP ▼	192.168.1.201	9	☒
cam1	80	Both ▼	192.168.1.199	80	☒
cam2	81	Both ▼	192.168.1.198	1024	☒
nas storage	443	TCP ▼	192.168.1.3	443	☒
transmission	80	TCP ▼	192.168.1.3	9091	☐
transmission	51413	Both ▼	192.168.1.3	51413	☒

eMule er et frit og gratis fildelings-program

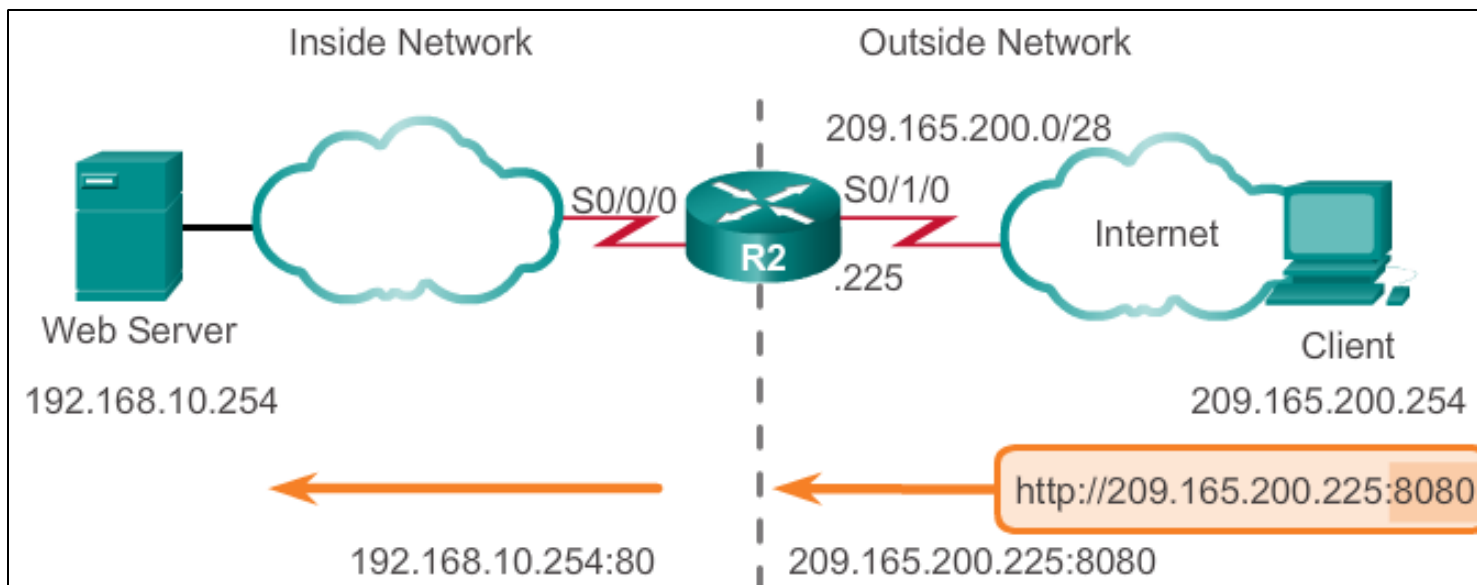
Wake-on-LAN (WOL) tænder en PC vha en netværks meddelelse – kan f.eks. sættes til at tænde hver dag kl. 8.

Network-attached storage (NAS) er diskplads på nettet

Port 51413 bruges almindeligvis til BitTorrent - en fildelings tjeneste hvor store filer hentes i mindre bidder fra flere forskellige steder.

Port Forwarding via IOS

Port forwarding i IOS'en er grundlæggende en statisk NAT oversættelse med et specificeret TCP eller UDP port nummer.



Establishes static translation between an inside local address and local port and an inside global address and global port.

```
R2(config)# ip nat inside source static tcp 192.168.10.254 80  
209.165.200.225 8080
```

Identifies interface serial 0/0/0 as an inside NAT interface.

```
R2(config)# interface Serial0/0/0  
R2(config-if)# ip nat inside
```

Identifies interface serial 0/1/0 as the outside NAT interface.

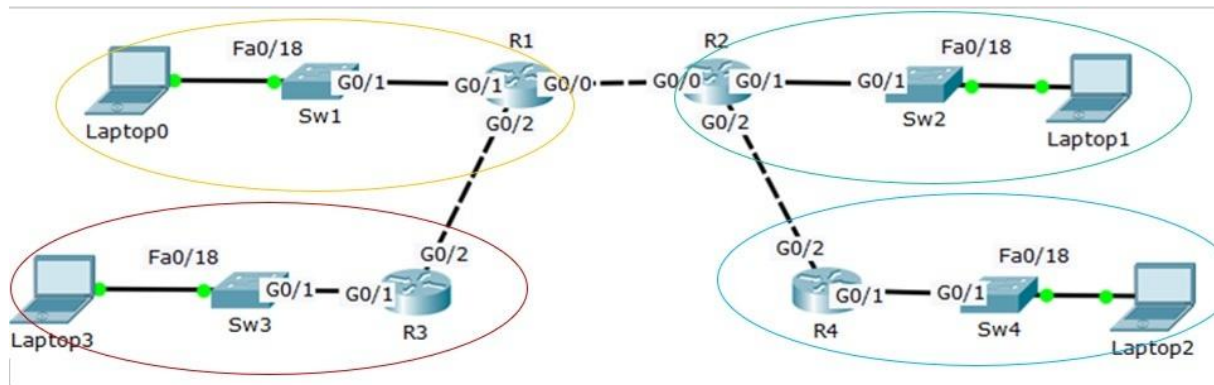
```
R2(config)# interface Serial0/1/0  
R2(config-if)# ip nat outside
```



Så kommer der en opgave, som bl.a. indeholder ACL og NAT 😊

Der er sikkert også udfordring nok til en dag ?

PacketTracer - Arbejd videre på OSPF opg. og læg NAT på.



1. Tilføj en router (ISP), vi leger den er Internettet. Opret et Loopback interface på ISP routeren og giv det ip adressen 5.5.5.5.
2. Forbind ISP til R4s G0/0. Brug 10.10.10.0/30 netværket mellem dem. Konfigurer en default static route på R4, der sender ukendt trafik til ISP. Udbred default static routen via OSPF til de andre routere. Dokumenter med skærmdump at default routen udbredes (fx vha `#sh ip route` på R3).
3. Konfigurer static routing på ISP, så der kan sendes svar retur til alle vores netværk.
4. Konfigurer NAT på R4, så al trafik fra enheder på R1s og R3s LAN får oversat afsender adresse til den adresse der er på R4s G0/0 interface, når trafikken passerer ud af R4 mod Internettet (ISP)
5. Trafik fra enheder på R2s og R4s LAN skal ikke have oversat afsender adresse, når trafik passerer gennem R4, ud til Internettet.
6. Dokumenter (med skærmdumps) at NAT virker og at du fra alle LANs kan ping 5.5.5.5.
7. Aflever PT fil + dokumentation/skærmdumps